

기존 방식: 코드에서 끝난다

기획 → 설계문 작성 → AI에게 개발 요청. 산출물은 코드 하나이고, 되돌아오는 길이 없다.



1 AI의 역할은 코더

판단은 사람이 하고, AI는 시킨 대로 코드만 쓴다. 왜 그렇게 만들었는지는 남지 않는다.

2 과정이 산출물이 아니다

스펙·계획·검증 근거가 기록되지 않는다. 결과 파일만 남고 요구사항 충족 여부를 증명할 수 없다.

3 수정은 프롬프트 재요청

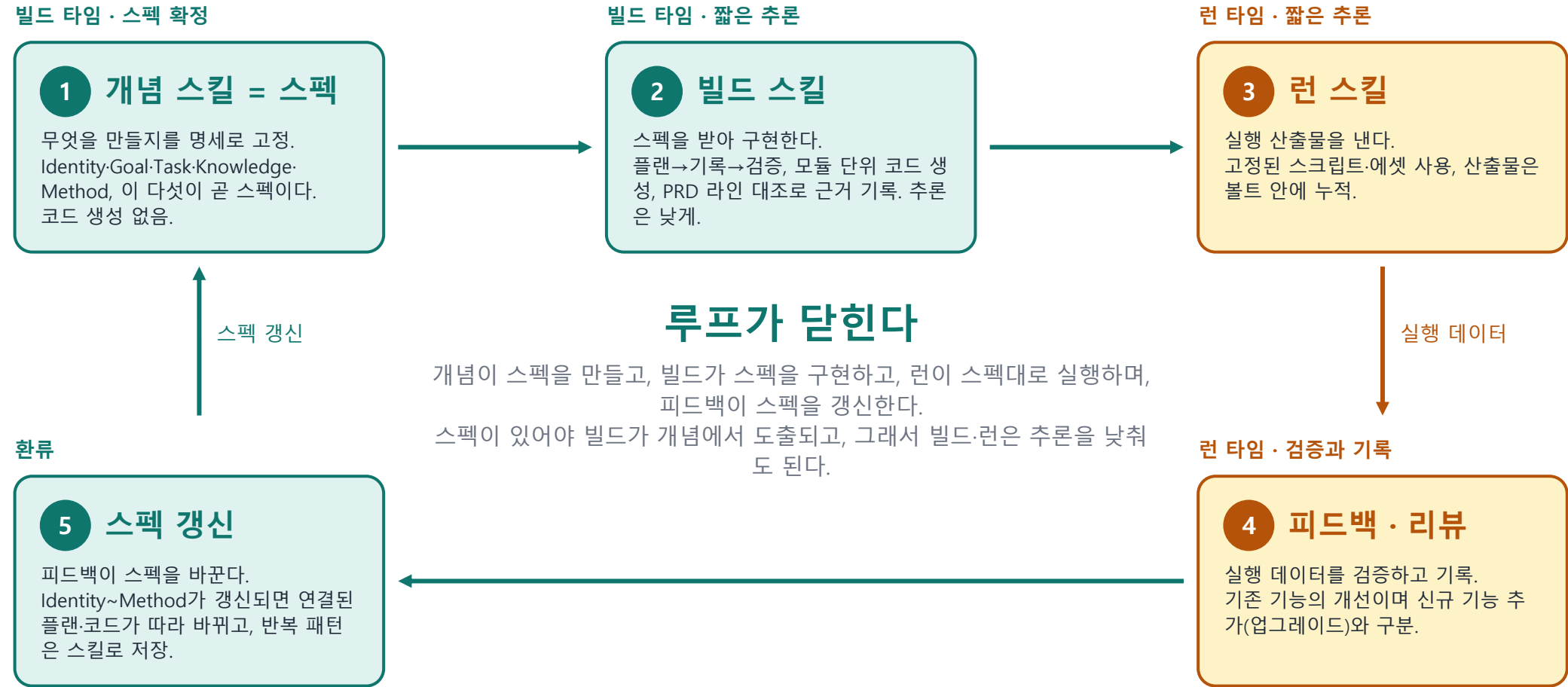
화면을 캡처해서 "이거 고쳐줘". 연관된 부분은 놓치고 걸만 고치는 반창고식 수정이 반복된다.

4 실행 데이터가 버려진다

실행 결과는 로그로 끝난다. 같은 문제를 다음 프로젝트에서 처음부터 다시 추론한다.

새로운 방식: 지식으로 되돌아온다

개념(스펙) → 빌드 → 런 → 피드백·리뷰 → 스펙 갱신. 마지막 단계가 다음 회차의 입력이 된다.



한눈에 비교

차이는 마지막 단계에서 생긴다. 코드에서 끝나는가, 지식으로 돌아오는가.

항목	기존 방식	지식행동사슬 방식
종착점	코드. 실행되면 프로젝트가 끝난다.	지식. 실행 결과가 검증·기록되어 다음 회차의 입력이 된다.
명세	설계문. 사람이 읽는 문서라 기계가 검증하지 못하고, 한 번 넘기면 갱신되지 않는다.	Identity·Goal·Task·Knowledge·Method가 곧 스펙. 빌드의 입력이 되고 피드백으로 갱신된다.
AI의 역할	코더. 설계문을 받아 시킨 대로 작성한다.	판단 주체. AI가 판단하고 스크립트가 반복 실행하며 스킬이 둘을 묶는다.
과정 기록	없음. 결과 파일만 남는다.	스펙·플랜·검증·근거(PRD 라인 대조)가 산출물의 일부다.
수정 방식	프롬프트 재요청. 보이는 함수만 고치고 연관 부분은 놓친다.	모듈 단위 피드백. 해당 모듈만 수정하고 연결된 스펙·플랜이 함께 갱신된다.
스크립트와 비용	매번 임시 스크립트 생성. 토큰이 새고 결과가 흔들린다.	고정·재사용. 패턴은 스킬로 저장되고 빌드는 소형 모델로도 충분하다.
보여주는 것	결과물 하나.	결과와 과정. 개념·빌드·런 세 가지를 함께 제시한다.

기존 방식의 반론("그냥 바이트 코딩이 빠르다")은 루프가 없을 때만 성립한다. 새 방식의 정당성은 이 루프가 실제로 닫히는지에 달려 있다.